NAME:- **FAROOQUI SHAHZEB AHMED**

ROLL NUMBER:- **SYA-29**

ASSIGNMENT:- **05**

AIM: **Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries**

Theory

1. **Scatter Plot:-** A scatter plot is used to display the relationship between two numerical variables. Each point represents an observation in the dataset. Useful for identifying correlation, trends, and outliers.
2. **Line Plot:-** A line plot connects data points using lines, typically used to show trends over time. Common in time series analysis.
3. **Bar Plot:-** A bar plot represents categorical data using rectangular bars. The height of the bar represents the value. Used for comparisons between categories.
4. **Histogram:-** A histogram shows the distribution of a numerical variable by dividing data into bins. Helps understand frequency distribution.
5. **Box Plot:-** A box plot displays data distribution using quartiles. It highlights median, outliers, and spread. Useful for detecting skewness and outliers.
6. **Pair Plot:-** A pair plot shows pairwise relationships between variables in a dataset. Useful for multivariate analysis.

Q1. **What is the importance of data visualization in data analysis? How does visualization help in exploratory data analysis (EDA)?**

ANS:- Data visualization helps in understanding patterns, trends, and relationships in data. It simplifies complex data into visual formats, making analysis easier.

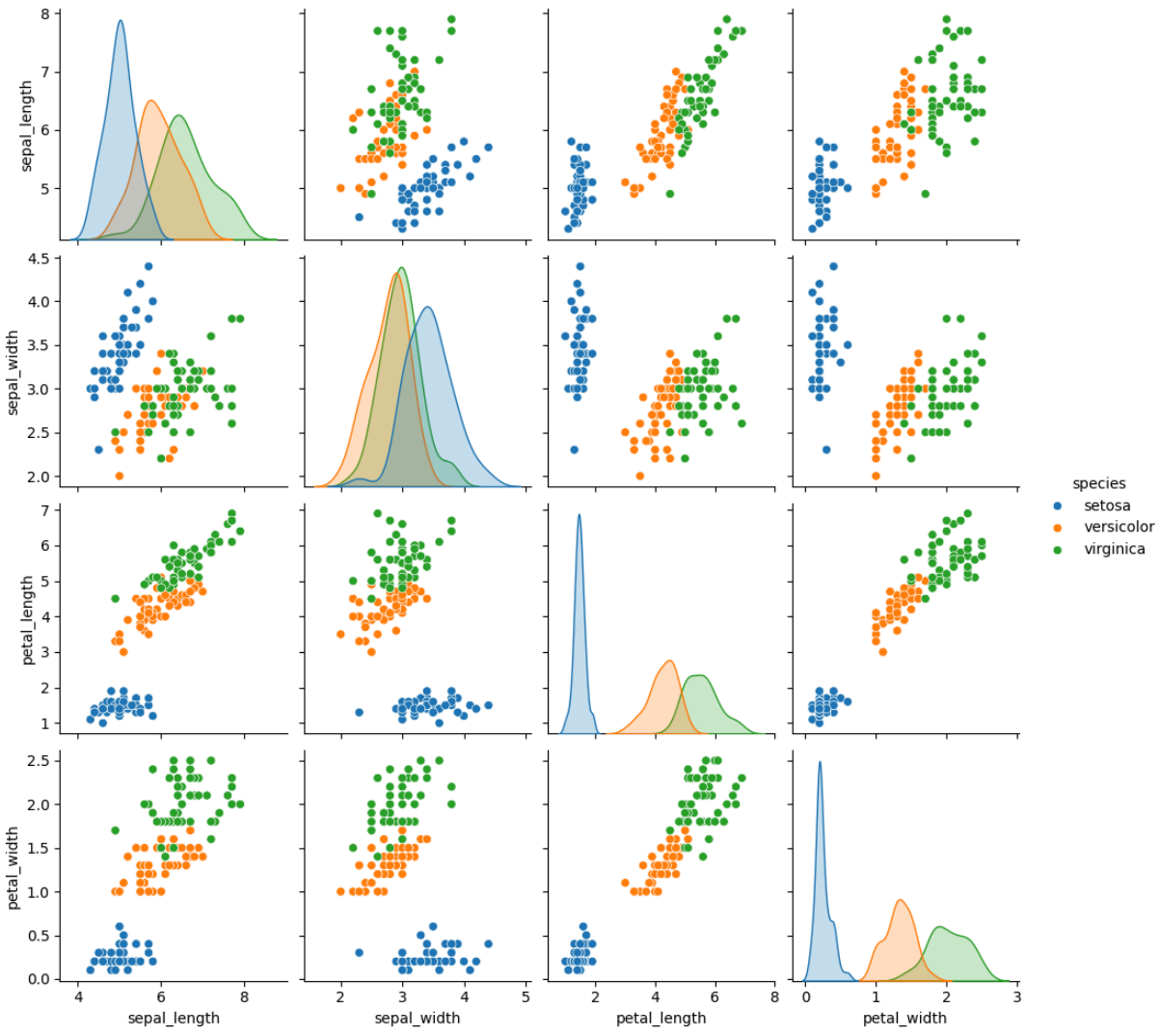
In EDA (Exploratory Data Analysis):

- Detects missing values and outliers
- Identifies patterns and correlations
- Helps in feature selection
- Improves decision-making

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('iris')

sns.pairplot(df, hue='species')
plt.show()
```



Q2. **What is the difference between Matplotlib and other visualization libraries?**

ANS:-

Q2. What is the difference between Matplotlib and other visualization libraries?

ANS:- Comparison of Matplotlib with Other Visualization Libraries

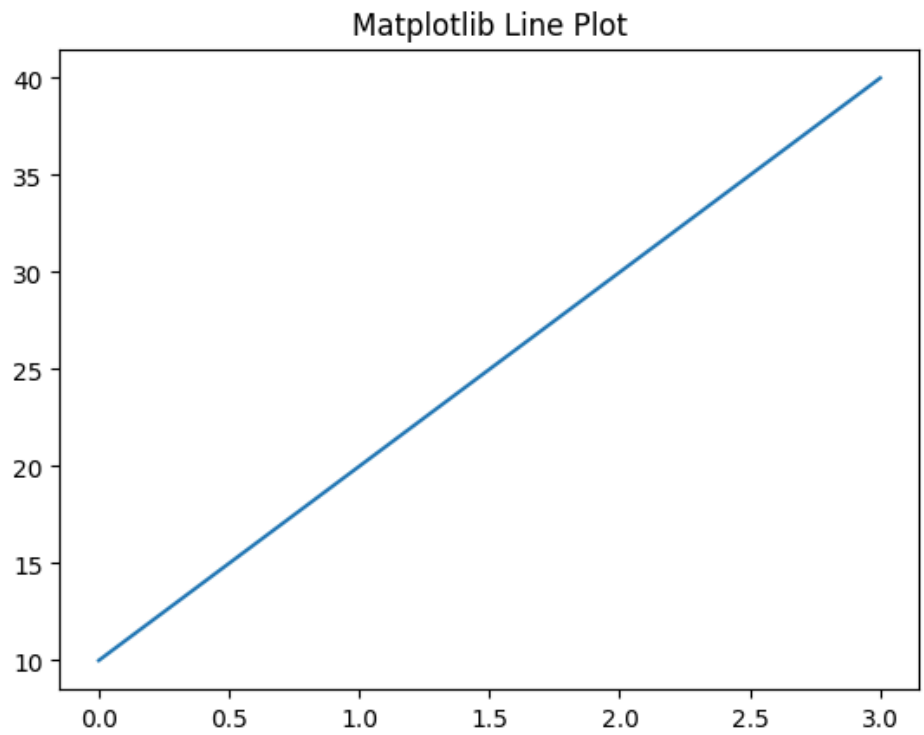
Feature/Library	Matplotlib	Seaborn
Primary Goal	Highly customizable, foundational plotting library	High-level interface for statistical graph
Ease of Use	Lower-level API, requires more code for complex plots	Higher-level API, easier for common sta
Interactivity	Limited interactivity (basic zoom/pan)	Limited interactivity (relies on Matplotli
Aesthetics	Basic, often requires customization for beauty	Beautiful, ready-to-use statistical plots
Use Cases	General-purpose plotting, fine-grained control, backend for other libs	Statistical data exploration, heatmaps,
Dependencies	NumPy	Matplotlib, SciPy, Pandas
Output Formats	Static images (PNG, JPG, PDF, SVG), basic interactive	Static images (PNG, JPG, PDF, SVG), ba
Learning Curve	Moderate to steep	Relatively low (if familiar with Matplotlib
Community Support	Very large, extensive documentation	Large, active

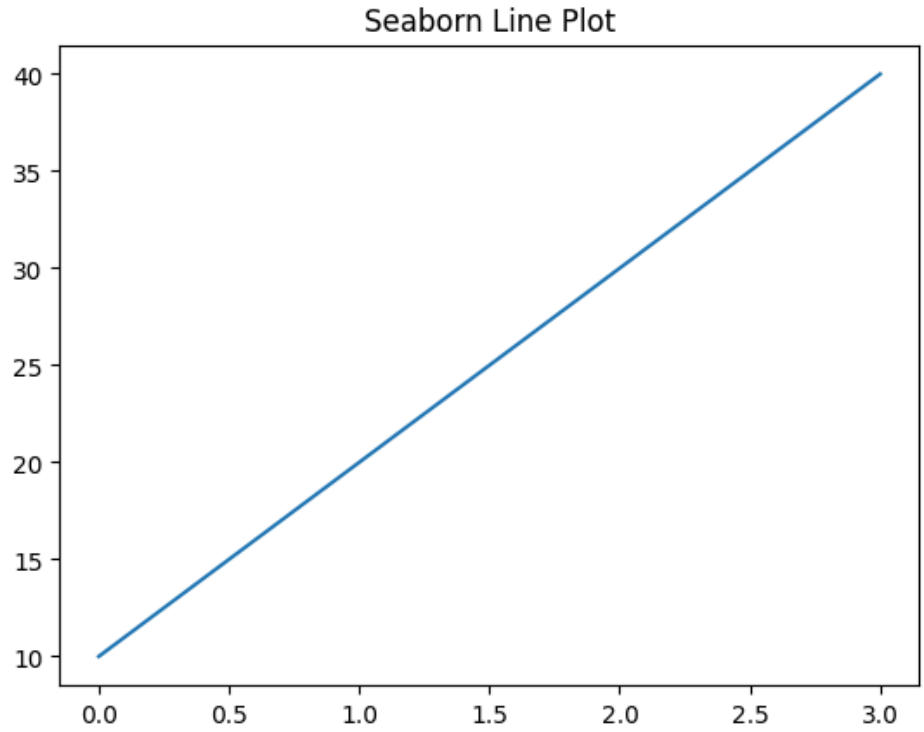
```
import matplotlib.pyplot as plt
import seaborn as sns

data = [10, 20, 30, 40]
```

```
plt.plot(data)
plt.title("Matplotlib Line Plot")
plt.show()

sns.lineplot(data=data)
plt.title("Seaborn Line Plot")
plt.show()
```





Q3. **What is the purpose of the pyplot module in Matplotlib?**

ANS:- pyplot is a module in Matplotlib that provides a simple interface for creating plots. It works like MATLAB and allows users to:

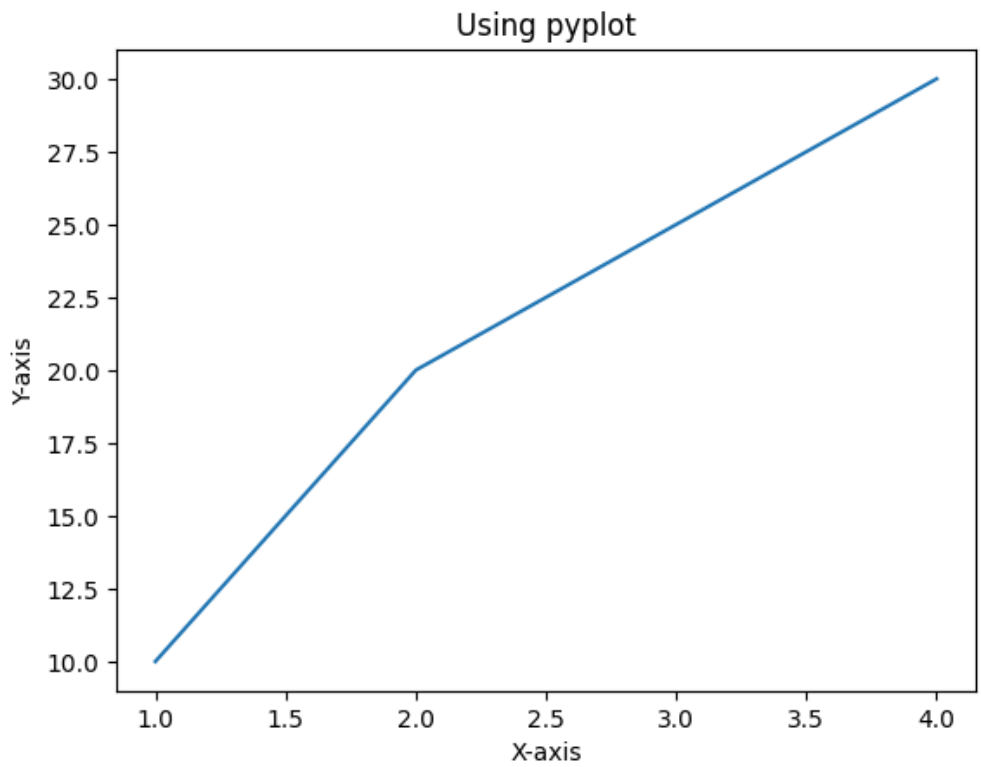
- Create figures and axes
- Plot graphs easily

- Customize plots

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Using pyplot")
plt.show()
```



Q4.**Explain the process of creating a basic line plot.**

ANS:- A basic line plot is a type of graph used to display data points connected by straight lines, typically to show trends or changes over a continuous variable such as time. In a line plot, the horizontal axis (x-axis) represents the independent variable, and the vertical axis (y-axis) represents the dependent variable. Each point on the graph corresponds to a data value, and the lines connecting these points help visualize the overall pattern or direction of the data. Line plots are commonly used to observe increases, decreases, or fluctuations in data over a period.

Steps:

1. Import matplotlib
2. Prepare data
3. Use plt.plot()
4. Add labels and title
5. Display using plt.show()

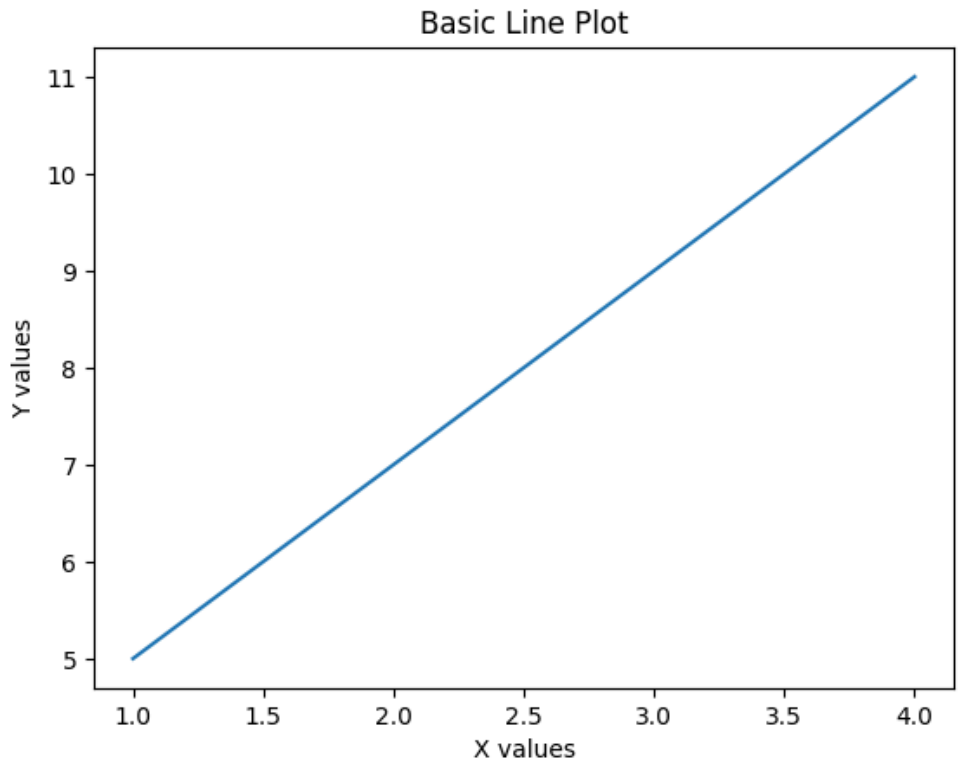
```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
y = [5, 7, 9, 11]

plt.plot(x, y)

plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Basic Line Plot")

plt.show()
```



Q5.**Why is customization important in data visualization? What customization options are available in Matplotlib?**

ANS:- Customization is an important aspect of data visualization because it enhances the clarity, readability, and overall effectiveness of a graph. A well-customized plot can highlight key insights, make important data points stand out, and improve the visual appeal of the chart. Without customization, graphs may appear plain or confusing, making it difficult for users to interpret the data correctly. Matplotlib provides a wide range of customization options such as changing colors, line styles, markers, fonts, figure size, and adding titles, labels, legends, and grids. Users can also adjust axis limits, add annotations, and modify background styles to better present the data. These customization features allow users to tailor the visualization according to the audience and purpose, ensuring better communication of insights.

Customization options in Matplotlib include:

- Colors (color='red')
- Line styles (linestyle='--')
- Markers (marker='o')
- Titles and labels
- Legends

- Grid
- Figure size

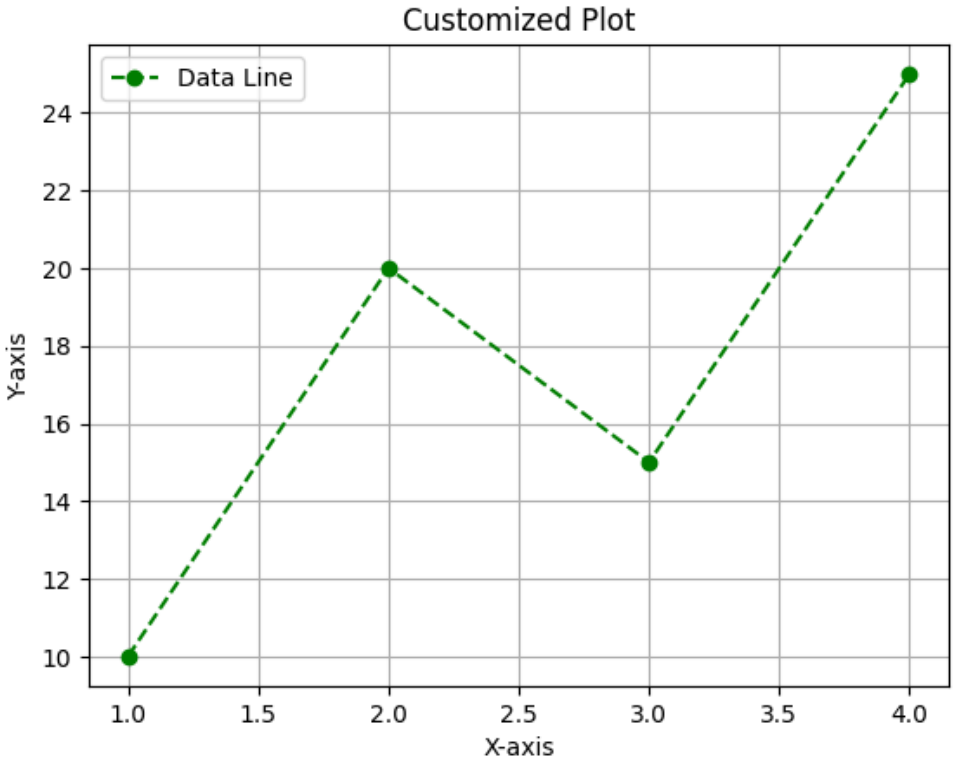
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

plt.plot(x, y, color='green', linestyle='--', marker='o', label='Data Line')

plt.title("Customized Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid()

plt.show()
```



CONCLUSION:- A basic line plot is a simple and effective way to visualize trends and relationships between variables, making it widely used in data analysis.